

RMS

```

1  #include <stdio.h>
2  int gcd(int a, int b) {
3      while (b != 0) {
4          a = a % b;
5          int temp = a;
6          a = b;
7          b = temp;
8      }
9      return a;
10 }
11 int lcm(int a, int b) {
12     return (a * b) / gcd(a, b);
13 }
14 int find_lcm(int arr[], int n) {
15     int res = arr[0];
16     for (int i = 1; i < n; i++) {
17         res = lcm(res, arr[i]);
18     }
19     return res;
20 }
21 void sort(int n, int burst[], int period[]) {
22     for (int i = 0; i < n - 1; i++) {
23         for (int j = i + 1; j < n; j++) {
24             if (period[i] > period[j]) {
25                 int temp;
26                 temp = period[i];
27                 period[i] = period[j];
28                 period[j] = temp;
29                 temp = burst[i];
30                 burst[i] = burst[j];
31                 burst[j] = temp;
32             }
33         }
34     }
35 }
36 int main() {
37     int n;
38     printf("Enter the number of processes: ");
39     scanf("%d", &n);
40     int burst[n], period[n];
41     printf("Enter CPU burst times:\n");
42     for (int i = 0; i < n; i++) {
43         scanf("%d", &burst[i]);
44     }
45     printf("Enter the time periods:\n");
46     for (int i = 0; i < n; i++) {
47         scanf("%d", &period[i]);
48     }
49     sort(n, burst, period);
50     int hyper = find_lcm(period, n);
51     printf("\nLCM = %d\n", hyper);
52     printf("\nRate Monotone Scheduling:\n");
53     printf("PID\tBurst\tPeriod\n");

```

```

55     for (int i = 0; i < n; i++) {
56         printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
57     }
58     float util = 0;
59     for (int i = 0; i < n; i++) {
60         util += (float)burst[i] / period[i];
61     }
62     float bound;
63     if (n == 1)
64         bound = 1.0;
65     else if (n == 2)
66         bound = 0.828427;
67     else if (n == 3)
68         bound = 0.779763;
69     else
70         bound = 0.693;
71     printf(
72         "\n%.6f <= %.6f => %s\n",
73         util,
74         bound,
75         (util <= bound) ? "true" : "false"
76     );
77     printf("Scheduling occurs for %d ms\n\n", hyper);
78     int remaining[n];
79     for (int i = 0; i < n; i++) {
80         remaining[i] = 0;
81     }
82     int last = -2;
83     for (int t = 0; t < hyper; t++) {
84         for (int i = 0; i < n; i++) {
85             if (t % period[i] == 0) {
86                 remaining[i] = burst[i];
87             }
88         }
89         int run = -1;
90
91         for (int i = 0; i < n; i++) {
92             if (remaining[i] > 0) {
93                 run = i;
94                 break;
95             }
96         }
97         if (run != last) {
98             if (run != -1) {
99                 printf(
100                     "%dms onwards: Process %d running\n",
101                     t,
102                     run + 1
103                 );
104             } else {
105                 printf("%dms onwards: CPU is idle\n", t);
106             }
107         }

```

```

107
108         last = run;
109     }
110     if (run != -1) {
111         remaining[run]--;
112     }
113 }
114 return 0;
115 }
116

```

OUTPUT

```

D:\1WA24CS231\RMS.exe
Enter the number of processes: 2
Enter CPU burst times:
20 35
Enter the time periods:
50 100

LCM = 100

Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35     100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

```

1WA24CS231

EDF

```
1  #include <stdio.h>
2  #include <limits.h>
3  int main() {
4      int n;
5      printf("Enter number of processes: ");
6      scanf("%d", &n);
7      int arrival[n], burst[n], deadline[n], period[n];
8      int remaining[n];
9      for (int i = 0; i < n; i++) {
10         printf(
11             "Process %d (arrival burst deadline period): ",
12             i + 1
13         );
14         scanf(
15             "%d %d %d %d",
16             &arrival[i],
17             &burst[i],
18             &deadline[i],
19             &period[i]
20         );
21         remaining[i] = burst[i];
22     }
23     printf("\nPID\tBurst\tDeadline\tPeriod\n");
24     for (int i = 0; i < n; i++) {
25         printf(
26             "%d\t%d\t%d\t%d\n",
27             i + 1,
28             burst[i],
29             deadline[i],
30             period[i]
31         );
32     }
33     printf("\nScheduling occurs for 20 ms\n\n");
34     int time = 0;
35     while (time < 20) {
36         int min_deadline = INT_MAX;
37         int selected = -1;
38         for (int i = 0; i < n; i++) {
39             if (arrival[i] <= time && remaining[i] > 0) {
40
41                 if (deadline[i] < min_deadline) {
42                     min_deadline = deadline[i];
43                     selected = i;
44                 }
45             }
46         }
47         if (selected == -1) {
48             printf("%dms : CPU is idle.\n", time);
49         } else {
50             printf(
51                 "%dms : Task %d is running.\n",
52                 time,
53                 selected + 1
```

1WA24CS231

```

54         );
55         remaining[selected]--;
56     }
57     time++;
58 }
59 return 0;
60 }
61

```

Output

```

D:\1WA24CS231\EDF.exe
Enter number of processes: 3
Process 1 (arrival burst deadline period): 0 2 4 5
Process 2 (arrival burst deadline period): 0 3 7 20
Process 3 (arrival burst deadline period): 0 2 8 10

PID      Burst   Deadline   Period
1         2       4          5
2         3       7          20
3         2       8          10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : CPU is idle.
8ms : CPU is idle.
9ms : CPU is idle.
10ms : CPU is idle.
11ms : CPU is idle.
12ms : CPU is idle.
13ms : CPU is idle.
14ms : CPU is idle.
15ms : CPU is idle.
16ms : CPU is idle.
17ms : CPU is idle.

```

1WA24CS231

Producer Consumer

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int mutex = 1;
4  int full = 0;
5  int empty = 5, x = 0;
6  int signal(int s) {
7      return (++s);
8  }
9  int wait(int s) {
10     return (--s);
11 }
12 void producer() {
13     empty = wait(empty);
14     mutex = wait(mutex);
15     x++;
16     printf(
17         "\nProducer has produced: Item %d",
18         x
19     );
20     mutex = signal(mutex);
21     full = signal(full);
22 }
23 void consumer() {
24     full = wait(full);
25     mutex = wait(mutex);
26     printf(
27         "\nConsumer has consumed: Item %d",
28         x
29     );
30     x--;
31     mutex = signal(mutex);
32     empty = signal(empty);
33 }
34 int main() {
35     int n;
36     while (1) {
37         printf("\n\nEnter 1.Producer 2.Consumer 3.Exit");
38         printf("\nEnter your choice: ");
39         scanf("%d", &n);
40         switch (n) {
41             case 1:
42                 if ((mutex == 1) && (empty != 0)) {
43                     producer();
44                 } else {
45                     printf("\nBuffer is full!");
46                 }
47                 break;
48             case 2:
49                 if ((mutex == 1) && (full != 0)) {
50                     consumer();
51                 } else {
52                     printf("\nBuffer is empty!");
53                 }
54             }
55         }
56     }
```

```
54         break;
55     case 3:
56         exit(0);
57         break;
58     default:
59         printf("\nInvalid Choice!");
60     }
61 }
62 return 0;
63 }
64 }
```

OUTPUT

```
Producer has produced: Item 1
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 1

Producer has produced: Item 2
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2

Consumer has consumed: Item 2
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2

Consumer has consumed: Item 1
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 1

Producer has produced: Item 1
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2

Consumer has consumed: Item 1
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
```

Dinning Philosopher

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosophers[N];
void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;
    while (1) {
        printf(
            "Philosopher %d is thinking.\n",
            id
        );
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf(
                "Philosopher %d picked up left fork %d.\n",
                id,
                left
            );
            pthread_mutex_lock(&forks[right]);
            printf(
                "Philosopher %d picked up right fork %d.\n",
                id,
                right
            );
        }
        else {
            pthread_mutex_lock(&forks[right]);
            printf(
                "Philosopher %d picked up right fork %d.\n",
                id,
                right
            );
            pthread_mutex_lock(&forks[left]);
            printf(
                "Philosopher %d picked up left fork %d.\n",
                id,
                left
            );
        }
        printf(
            "Philosopher %d is eating.\n",
            id
        );
        sleep(2);
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n",
            id,
```

```

        id,
        left,
        right
    );
}
return NULL;
}

int main() {
    int i;
    int ids[N];
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++) {
        pthread_create(
            &philosophers[i],
            NULL,
            philosopher,
            &ids[i]
        );
    }
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}

```

Output

```
"D:\1WA24CS231\Dinning Phil X + v
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 put down forks 4 and 0.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 4 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 2 picked up left fork 2.
Philosopher 0 picked up right fork 1.
```